

Uncertainty-Aware Underwater Semantic Segmentation for Robotics

A Deep Technical Analysis of the SUIM-Based Python/Colab Implementation

Technical Report

This report analyzes the uploaded implementation as executable software rather than treating its introductory prose as a substitute for the code. The analysis covers dataset handling, semantic-mask construction, preprocessing and augmentation, model construction, optimization, evaluation, calibration, Monte Carlo Dropout uncertainty, selective prediction, reproducibility, output packaging, and implementation-level limitations. No experimental values are invented; quantities that depend on notebook execution are described only as generated outputs.

Contents

Abstract	2
1 Introduction	2
2 Project Overview	3
3 Technical Foundations	4
4 Dataset and Input Processing	4
5 Train/Validation/Test Protocol	7
6 Preprocessing and Data Augmentation	7
7 Model Architecture and Implementation	7
8 Loss Functions and Class-Imbalance Handling	8
9 Optimization and Training Procedure	9
10 Evaluation Metrics	10
11 Held-Out Test Evaluation	11
12 Temperature Scaling	11
13 Monte Carlo Dropout Uncertainty	12
14 Uncertainty-Error Alignment and Selective Prediction	13
15 Qualitative Analysis Outputs	13
16 Per-Class Analysis	13
17 Execution Workflow	14
18 Implementation Architecture and Software Engineering Observations	14
19 Computational Considerations	15
20 Reproducibility and Reliability	16
21 Calibration, Uncertainty, and Robotics Interpretation	16
22 Limitations and Technical Considerations	16
23 Overall Technical Assessment	17
24 Conclusion	18

Abstract

The uploaded project implements a research-oriented semantic-segmentation pipeline for underwater imagery using the Semantic Segmentation of Underwater Imagery (SUIM) benchmark. Its primary model is a TorchVision DeepLabV3 network with a MobileNetV3-Large backbone and an eight-class output head. The implementation targets a Google Colab setting and combines pretrained image-classification features, image/mask integrity validation, deterministic dataset splitting, lightweight geometric and photometric augmentation, class-imbalance mitigation, mixed-precision training, validation-based checkpoint selection, probability calibration, and stochastic uncertainty estimation.

The central methodological focus is not segmentation quality alone. The code examines whether pixel-level uncertainty is aligned with segmentation errors and whether temperature scaling improves confidence calibration without changing predicted class labels. For uncertainty estimation, the implementation inserts explicit Dropout2d layers into the segmentation and auxiliary heads, keeps dropout active during stochastic inference, averages class probabilities over multiple passes, and derives predictive entropy, expected entropy, a mutual-information-style uncertainty proxy, and predictive variance. Calibration is performed by fitting a single positive temperature on a deterministic validation-pixel sample with L-BFGS and then applying the learned value to the held-out test set.

A detailed inspection also reveals several implementation distinctions. The executable dataset-preparation path operates on a pre-existing `/content/archive.zip`, despite introductory prose describing an official-download-first strategy. In addition, class-frequency weights are computed before the train/validation split, so validation labels affect the class-weight calculation. The flat SUIM-mirror branch also constructs a derived split rather than reproducing the official benchmark split. These points do not invalidate the project, but they are scientifically relevant when the pipeline is interpreted as a strictly isolated benchmark experiment.

1 Introduction

Semantic segmentation assigns a semantic category to every pixel in an image. In an underwater robotics setting, this capability is useful because a perception system may need to distinguish navigational background from divers, vegetation, wrecks, robotic equipment, reefs, fish, and seabed structures. The project therefore treats underwater perception as an eight-class pixel-labeling problem and adds an explicit uncertainty layer intended to indicate where the model may be unreliable.

The source addresses four research questions: the achievable segmentation quality of a practical DeepLabV3-MobileNetV3 model on SUIM, the relationship between Monte Carlo Dropout uncertainty and actual errors, the effect of temperature scaling on probabilistic calibration, and the implications of uncertainty for conservative robotic perception. These questions are implemented through a sequence of computational stages rather than a single model-training call.

The report focuses on what the executable source actually does. The source file is a Python script exported from a Google Colab notebook, and it contains both Markdown-style documentation strings and executable cells. Consequently, the analysis distinguishes conceptual documentation from code paths that are actually executed. The goal is to give the reader enough technical detail to reconstruct the computational logic without first opening the source file.

2 Project Overview

At a high level, the pipeline can be understood as seven linked stages:

1. environment setup and reproducibility configuration;
2. dataset archive validation, structure discovery, image/mask pairing, and canonicalization;
3. mask decoding, class-frequency analysis, train/validation splitting, and data loading;
4. DeepLabV3-MobileNetV3 model construction with explicit dropout;
5. supervised optimization under two loss settings and validation-based checkpoint selection;
6. deterministic test evaluation and temperature scaling;
7. MC-Dropout uncertainty evaluation, selective prediction, qualitative export, and result packaging.

The main software design principle is defensive preprocessing. Before training, the code checks the archive, extracts it with path-traversal protection, discovers alternative directory layouts, performs multiple levels of filename matching, validates files with Pillow, repairs image/mask resolution mismatches with nearest-neighbor interpolation, checks for exact cross-split image duplication, and only then builds data loaders.

The primary training experiment uses cross-entropy plus a soft multi-class Dice loss with a cross-entropy auxiliary head. A second experiment removes the Dice component and uses cross-entropy only. The same optimizer, image resolution, batch size, learning rate, augmentation policy, and validation criterion are used so that the loss formulation is the main intended difference.

Component	Implementation
Dataset	SUIM, eight semantic classes
Primary architecture	DeepLabV3 with MobileNetV3-Large backbone
Input resolution	256×256
Primary loss	Weighted cross-entropy + 0.6 Dice loss + 0.4 auxiliary cross-entropy
Optimizer	AdamW, learning rate 3×10^{-4} , weight decay 10^{-4}
Learning-rate schedule	Cosine annealing to 10^{-6}
Training budget	Up to 25 epochs with validation-mIoU early stopping
Stochastic inference	MC Dropout, 8 stochastic passes by default
Calibration	Scalar temperature fitted on validation pixels with L-BFGS
Uncertainty outputs	Predictive entropy, expected entropy, mutual-information-style uncertainty, predictive variance
Primary segmentation metrics	Pixel accuracy, per-class IoU, mIoU, per-class Dice, mean Dice
Confidence/risk metrics	Pixel ECE and AURC-like risk-coverage summary

Table 1: Key implementation choices exposed by the source code. Execution-dependent results are intentionally not reproduced here.

3 Technical Foundations

3.1 Semantic segmentation

The model receives an RGB image and returns a tensor of per-pixel class logits with shape (B, C, H, W) , where B is the batch size, $C = 8$ is the number of SUIM classes, and $H = W = 256$ after preprocessing. A softmax converts the logits to a categorical probability distribution at each pixel. The predicted class is the class with the maximum probability.

3.2 DeepLabV3 and MobileNetV3-Large

The implementation calls TorchVision’s `deeplabv3_mobilenet_v3_large` constructor with `num_classes=8` and `aux_loss=True`. Semantic-segmentation weights are not loaded. Instead, ImageNet-pretrained MobileNetV3-Large weights are requested for the backbone. This distinction matters: the segmentation head is initialized for SUIM, while the feature extractor can start from ImageNet representations. If the pretrained backbone weights are unavailable, the code explicitly falls back to a randomly initialized backbone.

DeepLabV3 uses atrous convolutions to preserve spatial resolution while expanding receptive fields and an atrous spatial pyramid pooling mechanism to capture context at multiple effective scales. MobileNetV3-Large reduces the parameter and compute burden relative to heavier backbones, which is consistent with the source’s Colab-oriented design goal.

3.3 Monte Carlo Dropout

The uncertainty mechanism is explicit. A Dropout2d layer with probability $p = 0.20$ is inserted immediately before the final segmentation classifier, and another is inserted before the auxiliary classifier. During stochastic inference, the complete model is first placed in evaluation mode and then only dropout modules are returned to training mode. This keeps Batch Normalization layers in evaluation mode while preserving stochastic dropout masks.

This procedure is a practical approximate Bayesian device. It provides a distribution of predictions induced by dropout perturbations, but it is not exact posterior inference. The source appropriately avoids presenting it as a perfect decomposition of epistemic and aleatoric uncertainty.

3.4 Probability calibration

The calibration stage uses a single scalar temperature $T > 0$. Given logits z_c for class c , calibrated probabilities are

$$p_T(c \mid x) = \frac{\exp(z_c/T)}{\sum_{k=1}^C \exp(z_k/T)}. \quad (1)$$

A positive scalar temperature preserves the ordering of logits, so the class selected by the argmax remains unchanged. What changes is the sharpness of the probability distribution. Values $T > 1$ soften probabilities, whereas values $0 < T < 1$ make them sharper.

4 Dataset and Input Processing

4.1 Dataset provenance represented by the code

The project identifies SUIM as an eight-class underwater semantic-segmentation benchmark. The source documentation states that the official resource contains 1,525 paired train/validation samples and 110 paired test samples, with the directory structure `train_val/images`, `train_val/masks`, `TEST/images`, and `TEST/masks`. The eight class names and RGB encodings are embedded directly into the implementation.

ID	Class	RGB code
0	Background / waterbody	(0, 0, 0)
1	Human divers	(0, 0, 255)
2	Aquatic plants / sea-grass	(0, 255, 0)
3	Wrecks / ruins	(0, 255, 255)
4	Robots / instruments	(255, 0, 0)
5	Reefs / invertebrates	(255, 0, 255)
6	Fish / vertebrates	(255, 255, 0)
7	Sea-floor / rocks	(255, 255, 255)

Table 2: Eight SUIM semantic classes represented by the implementation.

4.2 Archive handling and integrity validation

The executable dataset cell does not perform an online dataset download. It expects `/content/archive.zip`; if that file is absent, it searches the Colab `/content` directory for another ZIP and prefers names containing “SUIM”. This differs from the surrounding Markdown description, which states that an official Google Drive resource is tried first and a Kaggle mirror is retained as a fallback. The source therefore contains a documentation-to-implementation mismatch that should be acknowledged in any reproducibility description.

Once an archive is selected, the code performs several defensive checks. It computes a SHA-256 digest, verifies that the archive is recognized as a ZIP file, calls `testzip()` to detect corrupt members, and enumerates its entries. Extraction is performed through an explicit path-safety check: each archive member is resolved to an absolute target path and must remain below the extraction root before `extractall()` is allowed.

4.3 Directory discovery

Two layouts are supported. The first is the expected official-style layout, which is discovered recursively by normalizing directory names and searching for recognized train/test directories containing recognized image and mask child directories. The second is a flat mirror layout in which all images are held in a shared directory while masks are organized by class-specific folders such as BW, HD, PF, WR, RO, RI, FV, and SR.

The official-style branch preserves the discovered pairs and copies them into a canonical directory hierarchy. The flat-mirror branch reconstructs one semantic mask per image and then derives a train/test partition from sorted matched stems.

4.4 Robust image/mask matching

A notable engineering feature is the multi-stage matching strategy. For ordinary image/mask directories, matches are attempted in the following order:

1. exact lowercase stem;
2. removal of common mask suffixes such as `_mask`, `_label`, `_gt`, and `_segmentation`;
3. compact alphanumeric normalization;
4. a unique trailing numeric signature;
5. high-confidence fuzzy matching.

Fuzzy matching uses `difflib.SequenceMatcher`. A candidate is accepted only when the best similarity is at least 0.93 and exceeds the second-best similarity by at least 0.03, unless there is no second candidate. This margin condition is an explicit attempt to prevent ambiguous fuzzy matches from being silently accepted.

The flat-mirror branch follows a similar staged strategy, but the direction is reversed: each class-specific mask is matched to an image. Multiple class masks may point to the same image because an image can contain several semantic classes.

4.5 Semantic-mask reconstruction

In the flat-mirror case, the code constructs a single grayscale semantic mask from one or more class-specific binary masks. The output is initialized to class zero, and every nonzero location in a class mask is assigned that class identifier. In mathematical terms, for pixel position (i, j) and class-mask indicator $M_c(i, j) \in \{0, 1\}$, the implementation effectively performs ordered overwrite operations:

$$Y(i, j) \leftarrow c \quad \text{whenever } M_c(i, j) > 0. \quad (2)$$

Because masks are processed in ascending class-ID order, a later class can overwrite an earlier assignment if the source masks overlap. This is a concrete implementation assumption rather than a generic property of semantic segmentation, and it matters when evaluating a flat mirror whose class masks are not guaranteed to be perfectly disjoint.

4.6 Final pair validation and leakage checks

After canonicalization, image and mask stems are paired again and file readability is checked using Pillow’s verification method. If an image and its mask have different original dimensions, the mask is converted to RGB and resized to the image resolution with nearest-neighbor interpolation. This is the appropriate interpolation family for categorical segmentation labels because it avoids averaging class identifiers or colors.

The implementation also computes SHA-256 hashes for files and reports duplicate image or mask groups within each split. More importantly, it explicitly checks for exact image hashes shared between training and test partitions and removes training-side copies if such duplicates exist. A final assertion requires the post-cleanup overlap to be zero. This is a strong integrity feature, but it addresses exact image duplication rather than all forms of semantic duplication or near-duplicate imagery.

4.7 Mask decoding

The function `mask_to_class_ids` accepts either a two-dimensional integer mask whose values are already in $\{0, \dots, 7\}$ or a three-channel RGB mask matching the predefined SUIM color table. It also accepts unresolved grayscale RGB triplets whose common channel value is a valid class ID. Any remaining unknown color is treated as an error. The decoder therefore fails closed rather than silently mapping an unexpected color to a class.

5 Train/Validation/Test Protocol

The official 110-image test partition is kept separate from training and calibration. The source then splits the train/validation pool into a training subset and a validation subset using a fixed random seed and a validation fraction of 0.15.

The stratification label is not a multilabel representation of all classes present in an image. Instead, the code counts class pixels after removing the background class and uses the dominant remaining class as a single stratification label. This reduces severe split imbalance when possible while keeping the implementation simple. If stratified splitting fails, the code falls back to a deterministic unstratified split.

The flat-mirror branch requires at least 111 matched samples, and when at least 1,635 samples are available it creates 1,525 training samples and 110 test samples after lexicographically sorting the matched stems. This partition is explicitly reported by the code as a derived split, not the official benchmark split. When fewer than 1,635 samples are present, all validated samples are preserved while the final 110 are reserved for test.

A subtle but important experimental detail is that class-frequency weights are computed before this train/validation split. Therefore, validation masks influence the global class-frequency estimate used by the primary loss. This does not use validation predictions or validation performance for optimization, but it allows validation labels to affect a training hyperparameter. For strict train/validation isolation, the weight computation would ordinarily be based on training images alone.

6 Preprocessing and Data Augmentation

Images are resized to 256×256 using bilinear interpolation, while masks are resized with nearest-neighbor interpolation. The image is converted to a tensor and normalized using the standard ImageNet mean and standard deviation:

$$\mu = (0.485, 0.456, 0.406), \quad \sigma = (0.229, 0.224, 0.225). \quad (3)$$

For training samples, the implementation applies the following stochastic transformations:

- horizontal flip with probability 0.5;
- rotation with probability 0.35, using an angle uniformly sampled from $[-8^\circ, 8^\circ]$;
- color jitter with brightness 0.18, contrast 0.18, saturation 0.15, and hue 0.03.

The geometric transformations are applied identically to the image and mask, while color jitter is applied only to the image. This separation is methodologically appropriate because photometric changes should not alter semantic target labels.

The validation and test datasets disable augmentation and retain deterministic resizing and normalization. Data loaders use batch shuffling only for training.

7 Model Architecture and Implementation

7.1 Construction

The `build_model` function requests DeepLabV3 with a MobileNetV3-Large backbone, eight output classes, and an auxiliary loss head. The primary model is moved to the selected device.

A boolean attribute records whether ImageNet backbone initialization succeeded. The final primary classifier is modified by removing its final convolutional layer, inserting `Dropout2d(p=0.20)`, and appending the original final layer. The same operation is performed on the auxiliary classifier. This is a clear software-design choice: uncertainty is introduced at a specific, auditable location rather than relying on undocumented dropout already present somewhere in the network.

The source also creates a temporary probe model before training to inspect the number of parameters and verify the expected output tensor shape. The actual parameter counts are execution outputs and are not hard-coded into the report.

7.2 Forward-pass semantics

For an input batch X , the model returns a dictionary containing at least the primary output `out` and, because auxiliary loss is enabled, an `aux` output. During inference, only `out` is used for final predictions and uncertainty calculations. The auxiliary branch is therefore a training aid rather than part of the deployed prediction rule.

8 Loss Functions and Class-Imbalance Handling

8.1 Class-frequency weights

Let f_c denote the fraction of training-pool pixels assigned to class c . The source defines a smoothed inverse-frequency weight

$$w_c^{\text{raw}} = \frac{1}{\sqrt{f_c + 0.05}}. \quad (4)$$

and then normalizes the vector so that its mean equals one:

$$\tilde{w}_c = \frac{w_c^{\text{raw}}}{\frac{1}{C} \sum_{k=1}^C w_k^{\text{raw}}}. \quad (5)$$

These weights are used by the primary cross-entropy term. The square-root denominator makes the compensation less aggressive than direct inverse frequency, and the additive constant prevents extremely rare classes from receiving arbitrarily large weights.

8.2 Weighted cross-entropy

For logits $z_{n,c,i}$ and target class $y_{n,i}$, the weighted pixelwise cross-entropy used by the implementation can be represented conceptually as

$$L_{\text{CE}} = -\frac{1}{N} \sum_{n,i} \tilde{w}_{y_{n,i}} \log p_{n,y_{n,i},i}, \quad p_{n,c,i} = \text{softmax}(z_{n,\cdot,i})_c. \quad (6)$$

The exact normalization factor is governed by PyTorch’s cross-entropy implementation; the equation above describes the conceptual weighted objective rather than reproducing the framework’s internal reduction formula.

8.3 Soft multi-class Dice loss

The Dice term first converts logits to class probabilities and targets to one-hot tensors. For each class c , the implementation aggregates probabilities and target indicators over the complete batch and spatial dimensions. With smoothing constant $s = 1$,

$$D_c = \frac{2 \sum p_c y_c + s}{\sum p_c + \sum y_c + s}. \quad (7)$$

and the reported loss is

$$L_{\text{Dice}} = 1 - \frac{1}{C} \sum_{c=1}^C D_c. \quad (8)$$

The class averaging gives each class equal contribution to the overlap term, complementing the pixelwise nature of cross-entropy.

8.4 Auxiliary objective

The auxiliary output is supervised with the same weighted cross-entropy formulation. It is not given its own Dice component. The overall objective for the main experiment is therefore

$$L = L_{\text{CE}} + 0.6L_{\text{Dice}} + 0.4L_{\text{aux}}. \quad (9)$$

For the cross-entropy-only ablation, the main Dice contribution is removed and the auxiliary cross-entropy remains. The ablation path also does not pass the computed class weights because `train_experiment` sets `weights_to_use` to `None` for `ce_only`. Thus, the practical comparison is not only “with Dice” versus “without Dice”; it is also weighted CE in the primary configuration versus unweighted CE in the ablation configuration.

One implementation inefficiency is worth noting: `segmentation_loss` computes the Dice term even when `mode=ce_only`, although that value is not used in the main loss. This affects efficiency more than correctness.

9 Optimization and Training Procedure

Each experiment begins with its own deterministic, experiment-specific seed obtained from a CRC32-based function of the base seed and experiment identifier. The model is then trained with AdamW using

$$\eta_0 = 3 \times 10^{-4}, \quad \lambda_{wd} = 10^{-4}. \quad (10)$$

The learning rate follows cosine annealing with a minimum of 10^{-6} over the requested number of epochs. The default training budget is 25 epochs, with early stopping after 10 consecutive epochs without an improvement in validation mIoU.

During each training epoch, the code performs the following sequence:

1. moves images and targets to the selected device;
2. clears gradients with `set_to_none=True`;
3. performs a mixed-precision forward pass on CUDA when available;
4. computes the selected segmentation loss;
5. backpropagates the objective;
6. unscales gradients when AMP is active;
7. clips the global gradient norm to 1.0;
8. steps AdamW and the AMP scaler as appropriate;
9. accumulates a confusion matrix and loss statistics.

The validation pass is performed without optimization. After each epoch, the scheduler is advanced and the current state is checkpointed if validation mIoU exceeds the best value observed so far.

Experiment	Objective
Baseline: <code>experiment_01_baseline_ce_dice</code>	Weighted CE + 0.6 Dice + 0.4 auxiliary weighted CE
Ablation: <code>experiment_02_ablation_ce_only</code>	Unweighted CE + 0.4 auxiliary unweighted CE

Table 3: Training experiments encoded in the source.

The baseline checkpoint is subsequently used for calibration and uncertainty analysis. This separation matters because those downstream procedures are intended to operate on the primary trained model rather than on a model selected using test-set information.

10 Evaluation Metrics

10.1 Confusion matrix and segmentation metrics

The implementation accumulates a global $C \times C$ confusion matrix. For class c , true positives, false positives, and false negatives are

$$TP_c = CM_{cc}, \quad FP_c = \sum_r CM_{rc} - TP_c, \quad FN_c = \sum_r CM_{cr} - TP_c. \quad (11)$$

The class IoU is

$$\text{IoU}_c = \frac{TP_c}{TP_c + FP_c + FN_c}, \quad (12)$$

and the class Dice score is

$$\text{Dice}_c = \frac{2TP_c}{2TP_c + FP_c + FN_c}. \quad (13)$$

Mean IoU and mean Dice are arithmetic means over classes with nonzero union. Pixel accuracy is computed as the total number of correct pixels divided by the total number of evaluated pixels.

10.2 Expected calibration error

The code computes pixel-level ECE using 15 equal-width confidence bins. If B_m is bin m , the implementation follows the empirical structure

$$\text{ECE} = \sum_{m=1}^M \frac{|B_m|}{N} |\text{acc}(B_m) - \text{conf}(B_m)|. \quad (14)$$

Here, confidence is the maximum softmax probability at each pixel, and correctness is represented by a binary indicator.

This is explicitly a pixel-level calibration measure. It should not be interpreted as an independent-image calibration estimate because spatially adjacent pixels are strongly dependent.

10.3 Risk-coverage and AURC-like summary

The `risk_coverage` function sorts pixels by descending confidence, converts correctness to a binary error variable, and computes cumulative risk as the number of errors among retained pixels divided by the number retained. Coverage is the fraction retained. An area summary is then computed by trapezoidal numerical integration. Because the implementation directly integrates the sampled coverage points, it is best described as an AURC-like summary rather than assumed to implement every convention used by the broader selective-prediction literature. The deterministic test evaluation also records mean confidence, accuracy among the most-confident half of pixels, and average model inference time per test image.

11 Held-Out Test Evaluation

The source evaluates the saved checkpoints on the held-out test loader after model selection has already been performed from the training/validation split. For the baseline experiment, test logits are also collected and optionally subsampled to at most 100,000 pixels for calibration-related processing. The test evaluation does not refit the model or the temperature.

Per-experiment outputs include segmentation metrics, calibration metrics, confidence statistics, inference time, training time, optimizer identity, loss mode, and configuration parameters. Per-class IoU and Dice arrays are also written to separate metric files.

A noteworthy data-handling detail is that deterministic evaluation may collect baseline test logits even though the later temperature-fitting function independently collects validation logits. Thus, some of the test-logit collection is redundant from the perspective of temperature fitting. It does not, however, cause test labels to be used in the fitted temperature because the fitting routine itself operates on `val_loader`.

12 Temperature Scaling

Temperature fitting is performed exclusively on validation images. All validation logits are flattened from (B, C, H, W) into a matrix of shape (N, C) and paired with flattened target labels. If more than 100,000 pixels are available, a deterministic random subset is selected using a seed offset from the main seed.

Instead of optimizing T directly, the code optimizes an unconstrained scalar $\alpha = \log T$, then transforms it with

$$T = \exp(\alpha), \tag{15}$$

and clamps the resulting temperature to $[0.05, 20]$. The objective is validation cross-entropy:

$$L_{\text{temp}}(T) = -\frac{1}{N} \sum_{n=1}^N \log p_T(y_n | x_n). \tag{16}$$

The optimization uses L-BFGS with learning rate 0.5, at most 50 iterations, and strong-Wolfe line search.

After fitting, the same scalar temperature is applied to the held-out test logits. The resulting test ECE, AURC-like quantity, mean confidence, and segmentation metrics are compared with the unscaled test results. Because dividing all logits by a positive scalar does not change their order, the test class predictions should remain unchanged, while confidence values can change substantially.

The implementation therefore provides a clear conceptual separation: validation labels are used to learn a probability transformation, while test labels are used only to measure the final calibrated behavior.

13 Monte Carlo Dropout Uncertainty

For each image, the code performs $T = 8$ stochastic forward passes by default. Let $p_t(c \mid x)$ denote the softmax probability for class c at pixel x under stochastic pass t . The predictive mean is

$$\bar{p}_c(x) = \frac{1}{T} \sum_{t=1}^T p_t(c \mid x). \quad (17)$$

The final MC prediction is the class maximizing $\bar{p}_c(x)$, and predictive confidence is

$$q(x) = \max_c \bar{p}_c(x). \quad (18)$$

13.1 Predictive entropy

The predictive entropy is

$$H[\bar{p}(x)] = - \sum_{c=1}^C \bar{p}_c(x) \log(\bar{p}_c(x) + \epsilon), \quad (19)$$

with a numerical floor of 10^{-8} in the source.

13.2 Expected entropy and mutual-information-style uncertainty

The entropy of each stochastic prediction is computed first and then averaged:

$$\frac{1}{T} \sum_{t=1}^T H[p_t(x)]. \quad (20)$$

The source defines

$$U_{\text{MI}}(x) = H[\bar{p}(x)] - \frac{1}{T} \sum_{t=1}^T H[p_t(x)]. \quad (21)$$

This quantity is commonly associated with a mutual-information-style epistemic uncertainty signal under MC-Dropout approximations. The code clamps it to nonnegative values to suppress small numerical negatives.

13.3 Predictive variance

For additional stochastic spread information, the code computes the class-wise probability variance across MC samples and then averages the class variances:

$$U_{\text{var}}(x) = \frac{1}{C} \sum_{c=1}^C \text{Var}_t[p_t(c \mid x)]. \quad (22)$$

This is a separate dispersion measure; the project does not use it as the primary uncertainty-ranking variable.

14 Uncertainty-Error Alignment and Selective Prediction

The key uncertainty analysis asks whether high uncertainty is associated with actual mistakes. For every test pixel, the code stores mutual-information-style uncertainty, predictive confidence, and a binary correctness value.

A Spearman rank correlation is then computed between uncertainty and pixel error. A positive correlation means that pixels with larger uncertainty tend to have larger observed error values. The implementation also computes the empirical error rate in the top 10% most-uncertain pixels and in the bottom 10% least-uncertain pixels. Their difference is stored as an uncertainty-error gap.

For selective prediction, the code calls the existing risk-coverage function with $-U_{\text{MI}}$ as the confidence score. This reverses the ordering so that low-uncertainty pixels are retained first. In this framing, a useful uncertainty signal should allow the system to retain a high-confidence subset with lower residual error.

The source additionally evaluates sensitivity to the number of stochastic passes using $T \in \{4, 8, 12\}$ on at most six test images. Importantly, this sensitivity subset is taken from the beginning of the test dataset rather than randomly sampled. It is therefore a small diagnostic rather than a comprehensive estimate of MC-sample stability.

15 Qualitative Analysis Outputs

The source includes a qualitative export function that processes the first few stored MC records and creates visual products containing the input image, ground-truth mask, MC mean prediction, binary error map, predictive confidence, and mutual-information-style uncertainty. It also writes individual prediction, error, confidence, uncertainty, and uncertainty-overlay images.

These outputs are useful for inspecting whether uncertainty highlights boundaries, ambiguous regions, small structures, or other visually difficult areas. They are deliberately treated as diagnostic artifacts rather than scientific evidence by themselves. The present report does not reproduce these visual outputs, in accordance with the requested report constraints.

16 Per-Class Analysis

The baseline test evaluation creates a table containing class IDs, class names, IoU, and Dice score. This allows segmentation behavior to be inspected separately for each semantic category rather than relying only on a single macro-average.

This is particularly useful in an imbalanced underwater benchmark because pixel accuracy can be dominated by frequent background regions, while per-class IoU reveals whether less frequent object categories are being segmented effectively. The code’s mean IoU further provides a macro-style summary in which classes with valid unions contribute equally to the mean.

17 Execution Workflow

A reader can reconstruct the executable order as follows:

1. install the small external package requested by the source and import the numerical, imaging, PyTorch, TorchVision, and scikit-learn dependencies;
2. select CUDA when available, determine whether automatic mixed precision should be used, and establish a fixed base seed;
3. create output directories for checkpoints, metrics, tables, figures, predictions, uncertainty maps, logs, and configuration files;
4. select an existing ZIP archive, validate its integrity, and extract it using path-traversal checks;
5. detect either an official-style SUIM directory tree or a flat class-directory mirror;
6. construct or copy canonical image/mask pairs, then run final pairing, readability checks, dimension repairs, and exact cross-split leakage checks;
7. decode semantic masks, compute pixel-frequency statistics, and derive smoothed class weights;
8. split the train/validation pool into training and validation subsets and build the three data loaders;
9. probe the DeepLabV3-MobileNetV3 model and verify the output shape;
10. train the baseline and CE-only experiments with AdamW, cosine learning-rate decay, gradient clipping, AMP when available, validation-mIoU checkpointing, and early stopping;
11. evaluate both saved checkpoints on the held-out test split and write comparison tables;
12. fit a scalar temperature on validation pixels using the baseline model and L-BFGS;
13. evaluate raw and temperature-scaled confidence behavior on the held-out test split;
14. enable dropout stochastically and run multiple forward passes per test image to obtain uncertainty statistics;
15. measure uncertainty-error alignment, selective risk, MC-sample sensitivity, and per-class segmentation quality;
16. save configuration records, generated tables, visual diagnostics, a compact Markdown report, and finally a ZIP archive for Colab download.

18 Implementation Architecture and Software Engineering Observations

The file contains 51 function definitions and one dataset class. The functions naturally cluster into four layers: data acquisition and canonicalization, dataset verification and decoding, model/training utilities, and post-training analysis/packaging.

The dataset layer is unusually defensive for a compact notebook. It makes assumptions explicit through naming normalization, multiple matching passes, archive checks, and assertions. The model layer is modular, with a dedicated builder, loss function, epoch runner, experiment trainer, and checkpoint loader. The analysis layer is similarly modular, with separate deterministic evaluation, calibration, MC prediction, uncertainty evaluation, sensitivity analysis, and packaging functions.

There are, however, a few software-maintenance observations. The functions `stable_seed` and `seed_everything` are defined twice with the same implementation. Some imports such as `platform`, `subprocess`, and `math` are not material to the visible execution path. The function `build_stem_map` is defined but not used in the later pipeline, and `validate_official_dataset` is likewise not called by the final canonicalization path. These are minor code-hygiene issues rather than architectural defects.

Another sign of notebook evolution is repeated visualization code for the same training samples and a number of “UPGRADE” comments indicating incremental modifications. This is understandable in an experimental notebook, but it means that the source is best read as a research notebook with accumulated revisions rather than as a cleanly packaged software library.

19 Computational Considerations

19.1 Data preparation

Most dataset operations scale approximately linearly with the number of files. Exact and normalized dictionary-based matches are close to $O(N)$ in the number of images and masks. The fuzzy-matching stage is more expensive because it compares every unresolved item against the remaining candidates. In the worst case, this behaves quadratically in the number of unresolved files. In practice, the earlier exact and normalization passes are designed to resolve most matches before fuzzy matching is reached.

The duplicate-image check computes cryptographic hashes of image files and therefore requires reading each image at least once. The implementation prioritizes reliability over minimum I/O.

19.2 Model training

Training cost is dominated by DeepLabV3 forward and backward passes at 256×256 . The selected MobileNetV3-Large backbone reduces the burden compared with heavier backbones, while AMP can reduce memory consumption and accelerate computation on compatible CUDA hardware. Gradient clipping adds little computational cost relative to convolutional training.

19.3 MC-Dropout inference

The uncertainty stage is substantially more expensive than ordinary inference. A deterministic prediction requires one forward pass per image batch, whereas MC-Dropout requires approximately T forward passes for each image. With the default $T = 8$, the stochastic stage increases inference work by roughly the factor implied by the number of passes, although actual wall-clock cost depends on batch behavior and device utilization.

The uncertainty evaluator processes the test dataset one sample at a time rather than using batched MC inference. This simplifies storage of per-image stochastic results but is less computationally efficient than a vectorized batch implementation. The sensitivity study adds a small additional burden by repeating the same process for multiple MC sample counts.

19.4 Memory and numerical behavior

Probability tensors from MC inference are stored on CPU after each stochastic pass, so GPU memory pressure is controlled at the expense of CPU-side storage and host-device transfers. Entropy calculations use a small log floor to avoid $\log(0)$ behavior.

Temperature optimization is performed on validation logits moved back to the active device. With 100,000 calibration pixels and eight classes, the stored logits remain moderate in size, but the exact memory footprint depends on framework tensor representation and device configuration.

20 Reproducibility and Reliability

The source makes a substantial effort to create a reproducible record. It fixes a base seed, derives experiment-specific seeds, records Python and PyTorch versions, stores the selected device and GPU name, writes core hyperparameters to JSON, saves training histories, records the validation-fitted temperature, stores checkpoint metadata, and packages generated files into a ZIP archive.

The project also explicitly states that bitwise identity is not guaranteed across all Colab environments. This is a scientifically appropriate caveat. GPU kernels, library versions, hardware differences, data-loader behavior, and other low-level implementation details can introduce small differences even under fixed high-level seeds.

The reproducibility story would be strongest when the exact input archive is retained or independently identified by its SHA-256 digest. The implementation already computes this digest, which is useful evidence of dataset-instance identity, although the final reproducibility record shown in the source does not include the ZIP SHA-256 value itself.

21 Calibration, Uncertainty, and Robotics Interpretation

The project has a conservative robotics framing. Uncertainty is not treated as an automatic stop command. Instead, if empirical analysis shows that highly uncertain pixels have higher observed error rates, uncertainty can be used as a risk signal for downstream policies.

The source suggests operational consequences such as slowing down, increasing obstacle margins, requesting additional sensing, revisiting a viewpoint, or deferring object interaction. These are conditional design implications rather than claims that the benchmark experiment establishes deployment readiness.

Temperature scaling addresses probabilistic confidence on the evaluation distribution, but it does not make the model robust to sensor changes, unseen environments, or domain shift. Likewise, MC-Dropout uncertainty provides a stochastic approximation that may be informative without being universally trustworthy.

This distinction is central to interpreting the project scientifically: calibration, uncertainty estimation, and robust robotic decision-making are related but separate problems.

22 Limitations and Technical Considerations

22.1 Dataset scale and domain shift

The benchmark is small relative to many contemporary vision datasets. The official test set contains only 110 images, so estimates of uncertainty behavior and calibration can remain sensitive to sample composition. More importantly, an underwater model can encounter changes in camera characteristics, water properties, depth, illumination, color attenuation, turbidity, or sensor pipeline. Benchmark performance therefore cannot be equated with deployment robustness.

22.2 Approximate epistemic uncertainty

The MC-Dropout construction is an approximation. The mutual-information-style quantity is useful as a stochastic uncertainty indicator, but it should not be presented as exact Bayesian posterior uncertainty. The implementation itself makes this distinction explicit.

22.3 Aleatoric uncertainty

The notebook does not explicitly model aleatoric uncertainty. High entropy can arise because a scene is genuinely ambiguous, because the image is noisy, because the classes overlap visually, or because the model itself is uncertain. The project therefore does not claim a clean decomposition into uncertainty types.

22.4 Pixel dependence

ECE, risk-coverage, Spearman correlation, and error-rate comparisons are all computed at pixel level. Neighboring pixels are not independent observations. This is not a coding error, but it matters for statistical interpretation and especially for any future confidence-interval analysis.

22.5 Validation-label leakage through class weighting

As noted earlier, class-frequency weights are computed from the entire train/validation pool before the split. The validation labels therefore influence a training-side weighting choice. A stricter protocol would derive class weights only from the eventual training subset.

22.6 Derived flat-mirror split

The flat-mirror processing path does not reconstruct the official benchmark split from external metadata. It sorts matched stems and reserves contiguous ranges. Even when the counts are 1,525 and 110, the resulting partition must not be described as the official test split.

22.7 Dataset documentation mismatch

The Markdown documentation describes an official Google Drive-first download strategy with a Kaggle fallback, while the executable section shown in the source relies on an existing local ZIP and does not call the described download mechanisms. For a polished repository, this should be reconciled so that the documented reproduction path exactly matches the executable path.

22.8 MC sample sensitivity scope

The sensitivity study uses up to six test images taken from the beginning of the dataset. It is therefore useful as a diagnostic for sample-count behavior, but it is not a comprehensive stability study across the complete test distribution.

23 Overall Technical Assessment

The implementation is technically coherent and demonstrates a level of engineering maturity beyond a minimal segmentation training script. Its strongest aspect is the integration of several defensive layers: archive integrity checks, safe extraction, robust filename matching, semantic-mask validation, resolution repair with label-safe interpolation, exact cross-split duplicate detection, deterministic experiment seeds, validation-based checkpointing, explicit calibration fitting, and structured output packaging.

The model-training design is also internally consistent. The primary model has a lightweight backbone appropriate for a Colab-oriented workflow, class-frequency compensation is paired with Dice overlap optimization, the auxiliary classifier provides an additional supervised training signal, and the scheduler, gradient clipping, and AMP logic are integrated into a conventional PyTorch optimization loop.

The uncertainty component is particularly well framed conceptually. Rather than simply exporting uncertainty maps, the source measures whether uncertainty is statistically related to error and whether low-uncertainty subsets exhibit reduced risk. That makes the uncertainty study more informative than purely qualitative visualization.

The main scientific qualifications are explicit: the benchmark is small, uncertainty is approximate, calibration is distribution-dependent, pixel-level measurements have correlation structure, and the derived flat split is not the official benchmark split. The most important implementation-level caveat is the use of train/validation labels when computing global class weights, followed by the documentation mismatch around dataset downloading.

Taken together, the code provides a credible research software artifact for demonstrating competence in semantic segmentation, uncertainty-aware perception, calibration, reproducibility, and practical PyTorch engineering. Its strongest portfolio value comes from the way these pieces are connected into one auditable pipeline rather than from any single architectural novelty.

24 Conclusion

The uploaded project implements a complete uncertainty-aware underwater semantic-segmentation workflow centered on DeepLabV3 with a MobileNetV3-Large backbone and an eight-class SUIM label space. The software includes defensive dataset preparation, deterministic train/validation splitting, synchronized image-mask augmentation, class-imbalance weighting, a composite segmentation objective, AdamW-based optimization, validation-driven checkpoint selection, held-out test evaluation, temperature scaling, and MC-Dropout uncertainty analysis.

Mathematically, the core pipeline combines weighted cross-entropy, class-averaged soft Dice overlap, auxiliary supervision, categorical calibration through temperature scaling, entropy-based uncertainty, stochastic predictive variance, and selective risk analysis. The code connects these concepts directly to measurable quantities rather than leaving uncertainty as a purely visual concept.

From a software-engineering perspective, the project is structured as a reproducible Colab research workflow with saved checkpoints, histories, configuration records, metrics, diagnostics, and a final packaged archive. From a research perspective, it correctly separates model training from test-time calibration and uncertainty analysis and includes explicit limitations that prevent overclaiming deployment readiness.

The implementation is not free of caveats. In particular, the class-weight calculation precedes the train/validation split, the flat mirror may create a derived rather than official test partition, and the prose description of dataset acquisition does not perfectly match the executable archive-selection path. These are concrete, reviewable issues and should be treated as part of the technical record rather than silently overlooked.

Overall, the source constitutes a substantial research-oriented software project whose main contribution is an integrated, auditable workflow for studying the relationship among semantic-segmentation performance, probabilistic calibration, stochastic uncertainty, and conservative underwater robotic perception.

Source Traceability and Scope

This report is derived from the uploaded source file `suim_uncertainty_aware_semantic_segmentation_robotics(8).py`. The file is a Colab-exported Python representation of a notebook and contains approximately 5,200 executable/documentation lines. The report deliberately excludes generated figures, screenshots, plots, result visualizations, and execution-dependent numerical results. Where a statement concerns an implementation choice, it is grounded in the corresponding source code. Where the code permits more than one interpretation, the report uses qualified wording rather than asserting unverified author intent.

End of technical report